Measuring What a Failure-Routing Detector Adds: Representation, File Discovery and Denominators in AI/ML Python

Yufeiyang Chen*
School of Cyber Science and Technology, Sun Yat-sen University, Shenzhen, China
Correspondence: fei76161@gmail.com

3 November 2026

Abstract

Static detectors for exception handlers that turn a failed operation into an ordinary value are often compared with existing linters by counting the detector findings that no linter reports. That count mixes three things: the source forms each rule recognises, the files each tool actually analyses, and the set of findings used as the denominator. We describe a reproducible measurement protocol that separates them and apply it to an AST-based detector on eight pinned AI/ML Python packages (2,124 scanned files; 2,270 except handlers). Sixty-eight generated probes calibrate which selected Ruff, Bandit, Pylint and flake8 rules fire on which handler and suppression forms. In the corpus, none of the 19 contextlib.suppress candidates is co-located with a selected lint warning, or with an exception-related message from all 969 Ruff rules or Pylint's default set; a project-written Semgrep rule reaches 18. File discovery matters: Pylint silently skipped 152 files under directories without ___init___.py, and Semgrep 1.175.0 parses one file only partially. Replacing a name-based coverage mapping with the probe-calibrated one raises the predicted unmatched share from 18.3% to 47.1% with no change in detector output. A predicate refinement raises measured overlap from 41.1% to 52.9% while the number of co-located findings falls from 255 to 252: the smaller denominator explains the rise. Both candidate frames, the four-tool coverage rows and the probe matrix were re-executed from pinned inputs. The study measures warning coverage only. Exploratory LLM labels and curated maintainer records are reported with their provenance; they do not establish defect precision or recall.

Keywords: exception handling; static analysis; warning coverage; measurement validity; empirical software engineering; machine learning systems.

1 Introduction

An AI evaluator catches an API timeout and returns `0.0`. A document loader catches a decoding error and returns an empty string. In both cases the caller receives a value it could also receive after a successful computation. We call this conversion of an exception into an ordinary result *failure-routing*. Whether it is a defect depends on the caller's contract: a fallback may be intentional, or it may corrupt a score, a security verdict or a later computation. Swallowed and overly broad exception handling are well-studied problems [5, 7, 24], and existing linters already flag many of their forms.

A new detector for such handlers therefore has to answer a narrower question: *what does it report that existing tools do not?* The usual answer is a count of detector findings with no co-located linter warning. This paper argues that the count is a property of the whole comparison, not of the detector, and that it moves when any of four inputs moves:

*__Author contribution and AI assistance.__ The author originated the `failroute` idea and directed the study, including its research questions and key decisions. AI language tools assisted software development, testing, data analysis, and the drafting and revision of the text; the author takes responsibility for the final work. All corpus labels discussed in this paper were produced by LLM agents, not human annotators. **Relationships to the evaluated material.** The author wrote the detector under evaluation and the supplementary Semgrep rules; selected the main corpus partly from packages the author had contributed to; and submitted, from the author's GitHub account, the pull requests that form the curated maintainer records. See the conflict-of-interest statement.

1. the source *representation* each rule recognises (an `except` clause and a `contextlib.suppress` statement can express the same suppression);
2. the *files each tool actually analyses*, which need not be the files it was pointed at;
3. how *coverage is attributed*: by running the baselines, or by a mapping that predicts which rule would fire;
4. the *denominator*: which candidates enter the frame, and whether the unit is a finding, a source site or a handler.

Each of these can change the headline share without any change in what the detector finds or in whether the findings are defects.

We make the dependence explicit through a measurement protocol and apply it to `failroute`, an AST-based detector, on eight pinned AI/ML Python packages. The protocol keeps three kinds of evidence apart (Table 1): *behavioural calibration* of lint rules on generated probes, which establishes what a rule fires on; *tool-output coverage* on the corpus, which establishes which detector findings coincide with a linter warning; and *defect validity*, which would require independent adjudication of caller contracts. This paper reports the first two. For the third it reports only exploratory evidence and states why that evidence does not support precision or recall estimates.

Explicit handler

```
try:
    update_cache()
except Exception:
    pass
```

Suppress statement

```
with contextlib.suppress(
        Exception):
    update_cache()
```

Figure 1: Two representations of the same suppression. Ruff `SIM105` recommends rewriting the left form as the right one [2]. Selected Ruff `S110` and Bandit `B110` fire on the left form; none of the selected lint rules fires on the right form (Section 4.1). The example concerns warning reach, not whether the caller's contract permits suppression.

Research questions.

RQ1 *Representation.* Which selected lint rules fire on the handler and suppression forms the detector reports, on generated probes and in the corpus? (Section 4.1)

RQ2 *Read sets and attribution.* How much do the files each tool actually analyses, and the way coverage is attributed, change the measured unmatched share? (Section 4.2)

RQ3 *Denominator.* When the candidate predicate changes, how much of the change in measured overlap comes from the denominator rather than from the number of covered findings? (Section 4.3)

Section 5 then sets out what the available labels and maintainer records can and cannot add to these coverage results.

Contributions.

1. **A measurement protocol for detector increments** (Section 3): explicit units (finding, site, handler), a check of each tool's read set, probe-based calibration of rule semantics with positive controls, paired candidate frames with a denominator decomposition, and a rule that partial analyses are reported as bounds.
2. **Measurements on a pinned corpus**: a representation boundary at `contextlib.suppress` that the tested lint configurations, including all 969 Ruff rules and Pylint's default set, do not reach; two file-discovery effects; a 28.8-point correction of the predicted unmatched share caused only by coverage attribution; and an overlap increase explained by a shrinking denominator.
3. **Executable artefacts with stated audit status**: frozen exports, a current reproduction entry point that writes only to new directories, and an explicit separation between what was re-executed, what is cited from earlier runs, and what could not be re-run (Appendix A).

We do not claim to be the first to detect exception swallowing or failure-to-value conversion. Exception Miner already detects handling anti-patterns in Python with AST queries [22]; Souza et al. analyse 942 manually validated Python exception-handling bugs [24]; and Rollout Cards treats the handling of failed runs and the choice of reporting rules as a reproducibility problem in agent evaluation [11]. Our contribution is the measurement of a detector's *warning increment* and of the conditions under which that measurement is valid (Section 2).

2 Related work and positioning

Exception-handling anti-patterns and bugs. Cabral and Marques [7], Sawadpong et al. [20] and de Pádua and Shang [4, 5] established exception handling as an empirical object, including the prevalence of handling anti-patterns. Exception Miner identifies such anti-patterns in Java, TypeScript and Python with static AST queries and was evaluated on eleven multi-language projects [22]. Souza et al. [24] analysed 942 manually validated Python exception-handling bugs from 550 projects, including machine-learning and AI software, and derived root-cause and repair taxonomies whose categories include swallowing, overly broad handling and incorrect recovery. These studies characterise the phenomenon more broadly than our corpus does and, unlike our labels, rest on manual validation. We take the phenomenon as given and ask a different question: how a detector's incremental warning coverage over existing linters should be measured, and how sensitive that measurement is to representation, read sets, attribution and denominators.

Static-analysis warnings and their evaluation. Developers' use of static analysis depends on noise and actionable feedback [10, 12, 16, 25]. Tricorder and the FindBugs fixit describe filtering and prioritisation in deployed analysis [3, 17, 18]; work on false-positive mitigation and warning relevance gives further context [8, 15]. Hu et al. [9] study developers' suppression of static warnings. A warning-suppression annotation and a runtime `contextlib.suppress` statement operate at different layers: the latter changes exceptional control flow. Our probes measure which tested rules reach that runtime construct. We do not evaluate actionability, and we make no claim of superiority over other exception-handling tools.

Reporting rules and denominators. Rollout Cards [11] audits agent training and evaluation repositories and shows that reporting rules, including how failed, errored or skipped runs are dropped or counted, can change reported scores. It is the closest prior work in spirit: it already treats failed computations that enter a score, and the choice of denominator, as reproducibility problems. Our setting is different, a comparison between static detectors, but the lesson transfers: a coverage share is only interpretable together with its numerator, its denominator and the record of what each tool actually analysed.

LLM support and ML-system reliability. Souza et al. [23] evaluate prompted LLMs on four Python exception-handling tasks against a curated benchmark. Here LLM readings are retained exploratory labels with unrecoverable model provenance; they are not a benchmark. Pipeline coupling and boundary checks motivate reliability work in ML software [1, 6, 13, 14, 19, 21]; our corpus is drawn from that domain but our claims are about measurement, not about ML systems in general.

3 Measurement protocol

3.1 Units and the three kinds of claims

Units. A *finding* is one output record of the detector. A *site* is a source coordinate (package, file, line). A *handler* is an `except` clause in the AST. These are not interchangeable:

- one handler can produce several findings: the assignment channel of `silent-fallback` emits one finding per assigned target, so four sites carry two findings in each frame studied here (621 findings on 617 sites; 476 on 472);
- a `contextlib.suppress` statement produces a finding but is not an `except` handler (19 such sites in the 621-finding frame, 16 in the 476-finding frame).

We therefore report "findings per 100 `except` handlers" only as a ratio of tool output to handler count. It includes suppress findings and repeated outputs and is *not* the percentage of handlers affected. In the main corpus (2,270 handlers) the refined frame gives 476 findings, or 21.0 per 100 handlers, while 456 distinct handlers (20.1%) carry at least one finding; the 621-finding frame gives 27.4 findings per 100 handlers and 598 handlers with a finding (26.3%). Handler counts are from a direct AST count over the scan roots.

Co-location. A finding is *co-located* with a baseline when that baseline reports one of its selected rules in the same file within ±1 line, and *unmatched* when no baseline does. Co-location asks whether a tool

flagged the same place, not whether it expressed the same property: a broad-except warning can coincide with a failure-routing finding for a different reason. Coverage results below are co-location counts and nothing stronger.

Table 1: Three kinds of claims, the evidence each requires, and what this study provides.

Claim	Question	Evidence required	In this study
Behavioural calibration	On which handler and body forms does a lint rule fire?	executed rules on generated probes, with positive controls	68 probes, all re-executed (Sec. 3.5)
Tool-output coverage	Which detector findings coincide with a selected lint warning in a corpus?	executed baselines on the read set, per-finding rows	621- and 476-finding frames, re-executed
Defect validity	Is a finding a contract violation; what are precision and recall?	independent contract adjudication, held-out sample	**not measured**; exploratory LLM labels only (Sec. 5)

3.2 Detector and candidate frames

`failroute` parses each file to an AST and applies the rules of Table 2 through a shared registry. Five rules concern a caught failure that is suppressed or routed into a normal-looking value or continuation; `name-shadowing` is an adjacent exception-masking hazard. Rule membership describes a local construct, not a contract violation. Three gates remove a site before any rule runs: handlers catching only shutdown, iteration or cancellation exceptions; an opt-out comment; and per-project configuration. Whether a handler "records" the failure is decided syntactically (re-raise, exit, `warnings.warn`, or a logger-like call, with a deliberately stricter test for catch-all handlers). Appendix B gives the full predicates and their known asymmetries.

Table 2: Detector rules. Counts are findings in the 621-finding (v0.8.0) and 476-finding (v0.9.1) frames.

Rule	Shape	v0.8.0	v0.9.1
`silent-fallback`	handler returns or assigns a fallback constant without re-raising or recording	386	248
`no-action`	`except ...: pass` (or equivalent inert body)	213	209
`silent-suppress`	`with contextlib.suppress(...)`	19	16
`masked-exception`	catch-all that conditionally re-raises and otherwise falls back to a constant	3	3
`implicit-fallback`	handler falls through to an implicit `None`	0	0
`name-shadowing`	`except E as e:` rebinds `e` in the handler	0	0
Total		621	476

Two detector versions define two *candidate frames* on the same corpus. **v0.8.0** (tag `v0.8.0`) yields 621 findings. **v0.9.1** (tag `v0.9.1`) refines `silent-fallback`: it downgrades a fallback that the enclosing function's declared return type admits, records whether the value escapes the call or outlives it as object state, and attaches a severity and a static `covered_by` list predicting which lint rules fire on the handler. It yields 476 findings. The refinement was developed on this corpus and its labels, with no held-out set (Section 7).

3.3 Corpus and read sets

The main corpus consists of eight AI/ML packages fetched as pinned PyPI source distributions (Table 3). Each is recorded with URL, SHA-256, upload time and extracted-tree hash in `paper/corpus-lock.json`; `tools/fetch_corpus.py` verifies each archive and exits non-zero on mismatch. The eight sdists hold 2,354 `.py` files; the scan roots, which exclude tests, examples and vendored content outside the package body, hold 2,124 files, 524,229 lines and 2,270 `except` handlers. The packages were selected partly because of the author's contribution history; they are not a sample of AI/ML software.

Table 3: Pinned main corpus. Counts are inside each scan root.

Package	Version	Scan root	.py files	Lines	Handlers
inspect_ai	0.3.260	`src/inspect_ai`	755	198,351	859
pydantic_ai_slim	2.36.0	`.`	298	119,534	495
deepteam	1.0.9	`deepteam`	588	71,305	456
trl	1.12.0	`trl`	155	65,727	86
garak	0.16.0	`garak`	201	38,328	207
smolagents	1.26.0	`src/smolagents`	20	13,355	93
uqlm	0.6.5	`uqlm`	91	12,605	39
fickling	0.1.12	`fickling`	16	5,024	35
Total			**2,124**	**524,229**	**2,270**

A second, *style-opposite* control corpus of eight infrastructure packages (`attrs`, `httpx`, `requests`, `urllib3`, `click`, `pydantic`, `anyio`, `msgpack`; 263 files, 842 handlers, one bare `except`) is pinned in `paper/corpus-lock-control.json` and used in Section 5.3.

Read sets. Pointing a tool at a directory does not guarantee that it analyses every file below it. The protocol therefore records, for each tool, how it discovers files and how many of the 2,124 scan-root files it analysed (Table 4). This check exposed one omission that had affected earlier results and one partial analysis that bounds a supplementary result (Section 4.2).

Table 4: File discovery and read set per tool over the 2,124 scan-root files.

Tool	Discovery	Files analysed
`failroute`	own walker over `.py` files	2,124
`ruff`, `bandit`, `flake8`	filesystem walk	2,124; directory and explicit-file invocations give identical hits
`pylint` (directory argument)	walks packages; does not descend into a directory without `__init__.py`	1,972: 152 files silently not analysed (exit 0, no message)
`pylint` (file list; used for all results)	explicit file arguments	2,124
`semgrep` 1.175.0 (supplementary)	filesystem walk with default `.semgrepignore`	2,123 scanned (one `deepteam` test file ignored by default); one file only partially parsed

3.4 Baselines and matching

Each baseline is run on the same pinned scan roots with only the rules that address this family (Table 5); every report is reduced to (file, line) pairs and matched with ±1-line tolerance (`tools/coverage_union.py`). The four-tool union is the set of findings co-located with any of `ruff`, `bandit`, `pylint` and `flake8` with `bugbear`. A fifth, supplementary row uses a Semgrep rule set that the author wrote for the six shapes (`tools/semgrep-rules/exception-handling.yaml`); it shows what a syntactic pattern engine *can* express, not what users get by default, and it is reported separately from the four-tool union. Pylint's `W0703` is reported under its current identifier `W0718`.

3.5 Generated probes

Coverage can be attributed in two ways: by running the baselines on the corpus, or by *predicting* from a mapping which rules fire on a handler of a given form. The v0.9.1 detector ships such a mapping (`covered_by`). To make the mapping an empirical object rather than a reading of rule names, `tools/lint_mapping_probe.py` generates **68 probes**: eleven caught-type forms (bare; `Exception`; `BaseException`; `builtins.Exception`; narrow tuples; tuples mixing `Exception` with narrow, dotted or unresolvable members; single and dotted narrow names; a call expression) crossed with six body forms (`pass`, `...`, `continue`, returned constant, assigned constant, discarded comparison), plus a broad

Table 5: Baselines on the 621-finding frame. *Findings* is each tool's own output under the selected rules; *co-located* counts `failroute` findings within ±1 line. The Pylint output count depends on whether `click` is importable (563 with, 564 without); co-location does not.

Tool	Version	Rules	Findings	Co-located
`failroute`	0.8.0	all six	**621**	—
`ruff`	0.16.4	S110, S112	76	72
`bandit`	1.9.4	B110, B112	67	63
`pylint`	4.0.8	W0702, W0703/W0718, W0705, W0706	563/564	255
`flake8+bugbear`	7.3.0 / 25.11.29	E722, B001, B017	268	124
four-tool union				**255**
`semgrep` (project rules)	1.175.0	six shapes	—	+19

and a narrow `contextlib.suppress` statement. All four baselines run over the probe file, each hit is attributed to the probe whose function span contains it, and the resulting matrix is written to `bench/lint-rule-mapping.json`. Every selected rule has a positive control, so a selector that silently matches nothing fails the run. With `--check-parity` the script also compares the detector's predicted `covered_by` with the measured matrix, probe by probe.

The probes are a *behavioural calibration* of the comparison instrument. They show what each rule fires on; they say nothing about how often a form occurs in real code or whether a finding is a defect. They are also distinct from the detector's 68 unit fixtures (34 positive, 34 negative in fixture set v7), which test the detector against its own specification.

4 Results

4.1 RQ1: Representation

Probes. On the generated probes, `ruff S110/S112` fire on bare and catch-all handlers (including tuples containing `Exception` or `BaseException`); `bandit B110/B112` fire only on a bare handler or exactly `except Exception:`; and `S110/B110` fire on a `pass` body but not on `...`. Neither `contextlib.suppress` probe, broad or narrow, draws a message from any selected rule. The selected rule sets are therefore tied to the `except` representation.

Corpus. The 621-finding frame contains 19 `silent-suppress` findings, in five files of `inspect_ai` and `pydantic_ai_slim`. **None is co-located with a warning from the four selected baselines** (Table 6). Widening the baselines does not change this: running all 969 Ruff rules and Pylint's full default message set over those five files yields, within ±1 line of the 19 sites, only line-length messages (`E501`, `C0301`). The project's Semgrep rule co-locates with 18 of the 19, so the form is expressible by a syntactic rule. In the refined frame, `silent-suppress` falls to 16 findings, still with no four-tool co-location. Ruff's `SIM105` recommends rewriting `try/except/pass` into exactly this form [2]; a rewrite of that kind can preserve the suppression while removing it from the reach of the tested rules. Whether a given suppression is appropriate remains a contract question.

Table 6: Co-location by rule in the 621-finding frame. The Semgrep column adds the supplementary project rule; its value is the observed count of the 2026-11-03 diagnostic rerun, which matched the historical export (Section 4.2).

Rule	Findings	Four-tool union	With Semgrep	Unmatched (four-tool)
`silent-fallback`	386	180	181	206
`no-action`	213	72	72	141
`silent-suppress`	**19**	**0**	**18**	**19**
`masked-exception`	3	3	3	0
Total	621	255 (41.1%)	274 (44.1%)	366 (58.9%)

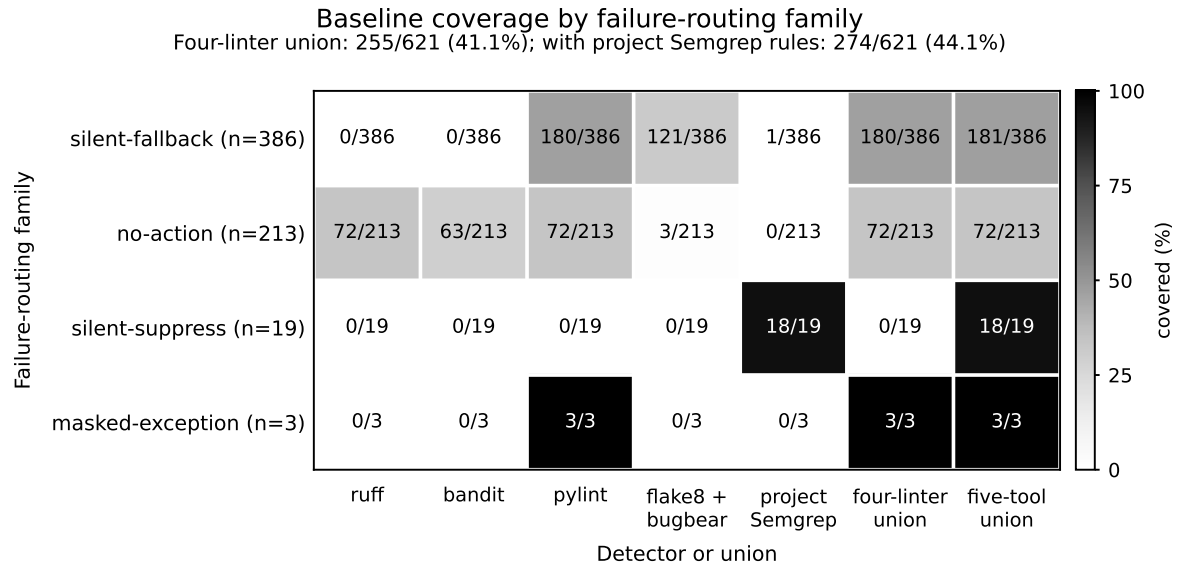


Figure 2: Co-location of the 621 findings by rule family and tool. Each cell gives co-located findings / family total. The four-linter union covers 255/621; adding the supplementary Semgrep rules gives 274/621 (upper bound 275, because one file was only partially parsed). `silent-suppress` is the family that no selected lint rule reaches.

Rule selection. The selection gives Pylint its broad-except rule (`W0718`) but not Ruff's analogue `BLE001`. Adding `BLE001` raises Ruff's own co-located count from 72 to 127 but leaves the union at 255, because every finding it newly reaches is already reached by `W0718`; the suppress family stays at 0/19. Per-tool comparisons therefore depend on the rule selection (Appendix C); the union does not, in this corpus.

4.2 RQ2: Read sets and coverage attribution

A read-set omission in the Pylint baseline. The original harness passed each scan root to Pylint as one directory argument. Pylint then skips every module under a directory without `__init__.py`, with exit status 0 and no message. In this corpus 152 of the 2,124 files (7.2%) sit under such directories (`inspect_ai` 123, `deepteam` 20, `garak` 9). Passing the file list instead recovers 34 selected messages in 20 of them and changes nothing on the files already analysed (0 messages lost, 0 gained). The four-tool union rises from 250 to 255 of 621 and the unmatched count falls from 371 to 366: the omission had *inflated* the detector's apparent increment. All results in this paper use the file-list invocation. Ruff, Bandit and flake8 return identical hits under both invocations; neither `--recursive=y` nor `--init-hook` recovers the skipped files.

A partial analysis in the supplementary baseline. Semgrep 1.175.0 reports a syntax error in `inspect_ai/model/_reasoning.py` at line 62, a `match` statement, and marks the file as partially analysed; it also skips one `deepteam` test module through its default `.semgrepignore`. Semgrep exits with status 0 and its console summary reads "Scan completed successfully" ("Parsed lines: ~100.0%"); the error is visible only in the JSON `errors` array and in `--verbose` output. The strict reproduction harness refuses such a run. A separate diagnostic (`tools/semgrep_diagnostic.py`) records the error and bounds its effect: the skipped test module contains no detector finding, and the partially analysed file contains one finding of the 621-finding frame (line 193, not co-located). The diagnostic observes 274/621 co-located findings with Semgrep added, identical to the historical export, with an upper bound of 275; for the 476-finding frame it observes 267/476, with no finding in the affected file. We therefore report the five-tool figures as an observed value with a bound, not as an error-free reproduction.

Predicted versus executed attribution. On the 476-finding frame, executed co-location leaves 224 findings (47.1%) unmatched. The original `covered_by` mapping, inferred from rule names, predicted

only 87 (18.3%). The probes identified why: the mapping credited `ruff S110` and `bandit B110` to every handler with an inert body regardless of what it caught, although both are broad-only; it treated `bandit`'s rules as if they fired on tuples and on `BaseException`, which they do not; and it did not distinguish a `pass` body from `...`. In the opposite direction, a tuple with an unresolvable member, such as `except (anyio.get_cancelled_exc_class(), Exception):`, was classified as narrow and lost the `W0718` and `BLE001` credits that do fire on it. For example, `garak/analyze/ci_calculator.py:40` is a literal `except (ValueError, TypeError): pass`; the old mapping called it covered, and all four baselines report nothing on that file.

Correcting the mapping removes credit from 146 findings (all `no-action`) and adds it to 3; the predicted unmatched count rises from 87 to 224, a correction of **28.8 percentage points**. The detector's output is the same 476 findings before and after, and the executed co-location is the same 252 covered and 224 unmatched: only the attribution changed. After the correction, the predicted mapping and executed co-location agree on all 476 findings. Five findings had initially disagreed; all five were in files that the directory-based Pylint invocation had not analysed, so the residual disagreement was a read-set effect, not a mapping effect. Thirty-six regression tests pin the corrected conditions; replaying them against a reconstruction of the pre-correction mapping (`tools/refix_replay.py`) fails 21 of them, so the tests discriminate between the two versions.

4.3 RQ3: Candidate predicate and denominator

Frame change. The v0.9.1 predicate reduces the frame from 621 to 476 findings (617 to 472 sites). The four-tool co-location *rises* from 255/621 = 41.1% to 252/476 = **52.9%**, while the unmatched count falls from 366 (58.9%) to 224 (47.1%). The refinement is not a pruning of the old frame: over distinct sites, 184 disappeared, 39 appeared and 433 persisted. Of the 184 dropped sites, 155 (84.2%) had been unmatched; of the 39 added, 15 (38.5%) were. The losses concentrate in `silent-fallback` (177 of 184, plus 4 `no-action` and 3 `silent-suppress`); all additions are `silent-fallback`.

Denominator decomposition. Write the overlap as $O = C/N$, with C co-located findings and N findings in the frame. Then

$$O_1 - O_0 = \underbrace{\frac{C_1 - C_0}{N_1}}_{\text{covered count}} + \underbrace{C_0 \left(\frac{1}{N_1} - \frac{1}{N_0} \right)}_{\text{denominator}}.$$

With $(C_0, N_0) = (255, 621)$ and $(C_1, N_1) = (252, 476)$, the denominator term contributes +12.51 percentage points and the covered-count term −0.63, giving the observed +11.88 points. **Overlap rises although fewer findings are co-located**; the increase is a property of the smaller frame. The decomposition describes a ratio change and does not attribute causal effects to individual predicate heuristics. Whether the smaller frame contains a higher share of defects is a separate question that the available labels cannot answer (Section 5).

Instrument versus predicate. RQ2 and RQ3 change the reported unmatched share by similar amounts through different mechanisms (Table 7). The attribution correction leaves all detector outputs fixed and changes the instrument; the predicate refinement leaves the instrument fixed and changes the candidates. Reporting either change as "the detector improved" would conceal which mechanism moved the number.

5 What the coverage results do not show

The coverage results establish where selected warnings reach and where they do not. They do not establish that an unmatched finding is a defect, or that a detector finds more defects than a simpler one. The project holds three further kinds of evidence; each is reported with its own denominator and none is pooled.

5.1 Exploratory LLM labels

A stratified random sample of 80 findings (rule × package strata, seed 20260831) was drawn from an earlier 659-candidate frame and labelled DEFECT, CONTRACT or FALSE POSITIVE by reading

Table 7: Paired measurements on the same pinned corpus. Each row changes one input and holds the others fixed.

Change	Held fixed	Unmatched before → after	What moved
Pylint read set (directory → file list)	detector v0.8.0, rules	371 → 366 of 621	tool read set
Coverage attribution (name-based → probe-calibrated)	detector v0.9.1, corpus	87 → 224 of 476 (predicted)	attribution only
Candidate predicate (v0.8.0 → v0.9.1)	baselines, matching rule	366/621 → 224/476	frame and denominator

the pinned source. **Both labelling rounds were run by LLM agents**, not human raters; they agreed on 80/80 items, which is compatible with shared model bias, and the model identity and version were not recorded. The labels were 12 DEFECT, 64 CONTRACT and 4 FALSE POSITIVE. All 12 DEFECT labels are in `deepteam`, all on bare `except:` handlers, and they fall into three copied code families; for five of them the failure-conversion mechanism is unestablished. A further 60 labels were added purposively, prioritising sites the refinement changed and sites no linter reaches (1 DEFECT, 58 CONTRACT, 1 FALSE POSITIVE); one rationale was later found to be incorrect and is recorded in `paper/annotation-corrections.json` without relabelling.

These labels served their useful purpose as a debugging instrument: the four false positives shared one control-flow bug in `implicit-fallback`, whose correction removed all ten such findings from the corpus (Appendix C). They **do not** measure precision: they are LLM judgements; the refinement was developed against all 140 of them; the sample no longer represents either current frame (72 of the 80 coordinates survive into the 621 frame); and the DEFECT labels are clustered in one package. Among the 104 labelled observations that join the refined frame, 12 of 48 co-located observations and 1 of 56 unmatched observations carry a DEFECT label. This describes a clustered, partly purposive set and supports no claim about incremental defect yield. A simple comparator illustrates the point: `flake8 --select E722` returns 134 bare-except candidates that include all 12 original DEFECT-labelled sites, whereas Ruff's `BLE001` (334 candidates) includes none of them. The full label tables and sensitivity analyses are in Appendix D.

5.2 Curated merged fixes

The historical ledger `paper/merged-pr-recall.csv` records 16 merged pull requests, submitted from the author's account, that had been classified as related to this pattern family. A source-linked audit on 2026-11-03 (`paper/reference-audit.json`; AI-assisted, not independent human adjudication) found that these changes do not share a common failure-routing construct: some are handler-local, some concern non-handler outcomes, and some are adjacent error-handling changes (for example, one adds a fallback where an exception previously propagated; another preserves exception identity through retry wrapping). We therefore **withdraw** the earlier pooled ratios (3/14 matched, 5/14 handler-reachable, 9/14 non-handler) rather than repair their denominator by assumption. Individual changes remain useful as concrete examples of failures outside a handler's local shape, such as a failed judge's NaN entering an aggregate score (`cvs-health/uqlm`#459). Appendix E summarises these and the broader maintainer records.

5.3 Control corpus

The main corpus contains 134 bare `except:` clauses, 131 in one package, which hold all 12 original DEFECT labels. To check whether the comparison depends on that style, the v0.9.1 detector and the same baselines were run on the style-opposite control corpus (Table 8). There, `flake8 E722` returns one candidate, a bare handler that re-raises. Of the four core packages' 243 handlers, the detector reports 38; a recorded code-reading pass over the 38 findings and the 91 unreported handlers without a `raise` classified none as a missed failure-routing defect. That pass was performed once, not by independent human adjudicators, and the 114 unreported handlers containing a `raise` were excluded without path-sensitive review. The control corpus also exposes cases that handler-local reasoning cannot

decide: a state flag consumed in a `finally` block is promoted to HIGH as if it were a substituted return value (`urllib3/connectionpool.py:812`), and an exception converted by a call that raises (`click/types.py:269`) is invisible to a callee-local rule.

Table 8: Main and control corpora under the same detector build (v0.9.1) and baselines. "Findings per 100 handlers" counts tool outputs, including suppress findings and repeated outputs; "handlers with a finding" counts distinct `except` handlers. Control-corpus readings come from one recorded code-reading pass.

	Main	Control core (4)	Control all (8)
`except` handlers	2,270	243	842
Bare `except:`	134	0	1 (re-raises)
Findings (per 100 handlers)	476 (21.0)	38 (15.6)	154 (18.3)
Handlers with a finding	456 (20.1%)	38 (15.6%)	153 (18.2%)
HIGH-severity findings	208	6	21
HIGH with empty `covered_by`	25	2	5
`flake8 E722` candidates	134	0	1

5.4 Summary of evidential status

Table 9 states, for each headline result, whether it was re-executed in the audit of 3–4 November 2026, is cited from an earlier recorded run, or could not be re-run. Nothing in the table is a defect-detection measurement.

6 Discussion

Coverage is a property of a comparison. The unmatched share depends on the representation each rule recognises, the files each tool analyses, how coverage is attributed and which candidates form the denominator. In this study each of them moved the headline figure while the detector's outputs, or the number of covered findings, stayed fixed or fell: a read-set omission by five findings, attribution by 28.8 points, and the denominator by 12.5 points. An unmatched finding is therefore an observation about one comparison, not evidence of an additional defect.

A protocol for detector increments. The study supports five concrete checks for future comparisons of failure-routing or exception-handling detectors:

1. *Lock inputs and verify read sets.* Pin the corpus by hash and record, per tool, how many target files were analysed; a silent exit status is not evidence of a complete scan.
2. *Calibrate rule semantics with executed probes.* Generate probes for every handler and body form the detector reports, including alternative representations, give every rule a positive control, and derive any predicted mapping from the measured matrix.
3. *Report numerator, denominator and unit.* State findings, sites and handlers separately; when the predicate changes, report site identities and absolute counts and decompose the ratio change.
4. *Report partial analyses as bounds.* Treat any tool error as an incomplete measurement and bound its effect rather than recording it as zero coverage or as success.
5. *Adjudicate contracts separately.* Defect precision and recall require independent contract adjudication on a sample kept apart from development, including findings from simple comparators such as `E722`.

Checks 1–4 are implemented and executed here. Check 5 is the missing evidence for any claim of incremental defect detection; a protocol for it is proposed in `docs/annotation-revalidation-protocol.md` but has not been executed.

Caller intent is the remaining validation problem. A return type may admit `None` or `False` without making an operational failure equivalent to a legitimate value; conversely, a constant fallback may be intentional. Constructed probes against the v0.9.1 build show both errors: a typed handler returning `False` on a database timeout, and an `Optional`-returning function returning `None` on a cache error, are missed, while a validation loop with a `return False` handler is reported at HIGH. Together

Table 9: Evidential status of the reported results (audit of 3–4 November 2026).

Result	Status	Note
8 pinned archives and extracted trees match the lock	re-executed	SHA-256 and tree hashes, fresh download
621-finding frame (v0.8.0) and 476-finding frame (v0.9.1, current)	re-executed	output files byte-identical to the frozen exports
Four-tool co-location, both frames (255/621, 252/476)	re-executed	per-finding files (1,097 rows) byte-identical, with both the current tools and commit `3b4c567`
Frame delta, denominator decomposition	re-executed	recomputed from the fresh rows; JSON equal apart from its timestamp
68-probe matrix, mapping parity, 21/36 replay	re-executed	matrix identical apart from the interpreter path
19 suppress sites vs. all Ruff rules and default Pylint	re-executed	only line-length messages nearby
E722 and BLE001 candidate counts	re-executed	134 and 334; JSON equal apart from its timestamp
Control-corpus scans	re-executed	8/8 packages match the committed exports
Semgrep five-tool co-location (274/621)	partial	observed 274, bound 275; one file partially parsed
Pylint directory-invocation counts (250/621)	cited	recorded at commit `3b4c567`; superseded
Control-corpus code readings; LLM labels; maintainer records	cited	single recorded passes; LLM provenance unrecoverable
Unit fixtures, set v7 (68 synthetic cases)	re-executed	34 positive and 34 negative agree; not real-code defect precision
Rebuilt figures and `intervals.json` vs. committed files	re-executed	same text and data; drawn geometry within stated tolerances
Claim checks needing unpublished commits or the companion repository	historical	8 claims, reported HIST; 5 re-derived on public commits
Defect precision or recall	not measured	requires independent adjudication

with the state-flag and call-conversion cases of Section 5.3, they show that local shape does not determine contract violation, which is why this paper restricts its claims to warning coverage.

7 Threats to validity

Construct validity. Co-location within one line does not establish that two tools diagnose the same property, and a farther warning may still be relevant. The supplementary Semgrep rules were written by the author and are reported separately and with a bound. The probes calibrate rule behaviour on synthetic code; they do not establish the frequency of each form in practice.

Internal validity. The v0.9.1 predicate and its `covered_by` mapping were developed on this corpus and its 140 labels, with no held-out set; the refined-frame results are descriptive of this development corpus. A harness omission and several mapping errors were found during the study and corrected (Appendix C); others may remain. The pre-correction mapping used by the replay is a reconstruction, not a recovered git object.

External validity. The main corpus consists of eight purposively selected packages, partly chosen from the author's contribution history; the control corpus of eight infrastructure packages represents one opposite style, not a population. All measurements describe pinned package and tool versions.

Label and record validity. All labels were produced by LLM agents whose model identity and version were not recorded; the label process cannot be reproduced, and the two rounds' agreement may reflect shared bias. The original annotation protocol does not describe the process actually run. The curated fixes and closed pull requests were submitted by the author, their family membership was revised by an AI-assisted audit, and the tracked-site dataset uses a weaker criterion than the detector. These records are therefore reported as exploratory and are not combined into any rate.

8 Conclusion

Measuring what a failure-routing detector adds over existing linters is a measurement problem before it is a detection problem. On eight pinned AI/ML packages, generated probes show that the selected lint rules do not reach the `contextlib.suppress` representation, and the corpus confirms that none of 19 such candidates is co-located with a selected warning. A Pylint read-set omission, a name-based coverage mapping and a change of denominator each moved the unmatched share materially, while the detector's outputs, or the number of covered findings, stayed fixed or fell. The protocol and artefacts make these effects executable and auditable. Whether the remaining unmatched findings are defects is a separate question that requires independent contract adjudication on held-out code, including findings from simple baselines.

Data availability

The detector, analysis scripts, pinned corpus locks, frozen exports, label files and audit records are available at `https://github.com/feiiiiii5/failroute` (MIT licence); the detector is also released on PyPI as `failroute`. The corpus is eight publicly distributed PyPI source distributions, each used under its own licence, and is re-fetched and hash-verified rather than redistributed. The maintainer records consist of public GitHub issue and pull-request records cited by number; no non-public data about identifiable individuals was used. Appendix A gives the reproduction commands.

Conflict of interest

The author developed the detector under evaluation and wrote the supplementary Semgrep rules. The main corpus was selected partly from packages to which the author had contributed, and the curated merged fixes and closed pull requests were submitted from the author's GitHub account. These relationships are also stated on the title page. To limit their effect, results unfavourable to the detector are reported alongside favourable ones, and no defect-detection claim is made.

Manuscript licence

This manuscript is licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). Code and third-party artefacts keep their own licences.

References

[1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300, 2019. doi: 10.1109/ICSE-SEIP.2019.00042.

[2] Astral. Ruff rule SIM105: suppressible-exception, 2026. URL https://docs.astral.sh/ruff/rules/suppressible-exception/. Documentation read 10 October 2026.

[3] Nathaniel Ayewah and William Pugh. The Google FindBugs fixit. In *Proceedings of the 19th international symposium on Software testing and analysis*, pages 241–252, 2010. doi: 10.1145/1831708.1831738.

[4] Guilherme Bicalho de Pádua and Weiyi Shang. Revisiting exception handling practices with exception flow analysis. In *2017 IEEE 17th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 11–20, 2017. doi: 10.1109/SCAM.2017.16.

[5] Guilherme Bicalho de Pádua and Weiyi Shang. Studying the prevalence of exception handling anti-patterns. In *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*, pages 328–331, 2017. doi: 10.1109/ICPC.2017.1.

[6] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 1123–1132, 2017. doi: 10.1109/BigData.2017.8258038.

[7] Bruno Cabral and Paulo Marques. Exception handling: A field study in Java and .NET. In *ECOOP 2007 — Object-Oriented Programming*, Lecture Notes in Computer Science, pages 151–175. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. doi: 10.1007/978-3-540-73589-2_8.

[8] Zhaoqiang Guo, Tingting Tan, Shiran Liu, Xutong Liu, Wei Lai, Yibiao Yang, Yanhui Li, Lin Chen, Wei Dong, and Yuming Zhou. Mitigating false positive static analysis warnings: Progress, challenges, and opportunities. *IEEE Transactions on Software Engineering*, 49:5154–5188, 2023. doi: 10.1109/TSE.2023.3329667.

[9] Huimin Hu, Yingying Wang, Julia Rubin, and Michael Pradel. An empirical study of suppressed static analysis warnings. *Proceedings of the ACM on Software Engineering*, 2:290–311, 2025. doi: 10.1145/3715729.

[10] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. Why don't software developers use static analysis tools to find bugs? In *2013 35th International Conference on Software Engineering (ICSE)*, pages 672–681, 2013. doi: 10.1109/ICSE.2013.6606613.

[11] Charlie Masters, Ziyuan Liu, and Stefano V. Albrecht. Rollout Cards: A reproducibility standard for agent research. arXiv:2605.12131v1 [cs.AI], version 1, submitted 12 May, 2026. URL https://arxiv.org/abs/2605.12131v1.

[12] Lisa Nguyen Quang Do, James R. Wright, and Karim Ali. Why do software developers use static analysis tools? A user-centered study of developer needs and motivations. *IEEE Transactions on Software Engineering*, 48(3):835–847, 2022. doi: 10.1109/TSE.2020.3004525.

[13] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55:1–29, 2022. doi: 10.1145/3533378.

[14] Neoklis Polyzotis, Martin Zinkevich, Sudip Roy, Eric Breck, and Steven Whang. Data validation for machine learning. In *Proceedings of the Machine Learning and Systems Conference (MLSys) 2019*, 2019. URL https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html.

[15] Louis-Philippe Querel and Peter C. Rigby. WarningsGuru: integrating statistical bug models with static analysis to provide timely and specific bug warnings. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 892–895, 2018. doi: 10.1145/3236024.3264599.

[16] Joseph R. Ruthruff, John Penix, J. David Morgenthaler, Sebastian Elbaum, and Gregg Rothermel. Predicting accurate and actionable static analysis warnings. In *Proceedings of the 30th international conference on Software engineering*, pages 341–350, 2008. doi: 10.1145/1368088.1368135.

[17] Caitlin Sadowski, Jeffrey Van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. Tricorder: Building a program analysis ecosystem. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, pages 598–608, 2015. doi: 10.1109/ICSE.2015.76.

[18] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspan. Lessons from building static analysis tools at Google. *Communications of the ACM*, 61:58–66, 2018. doi: 10.1145/3188720.

[19] Nithya Sambasivan, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, and Lora M Aroyo. "Everyone wants to do the model work, not the data work": Data cascades in high-stakes AI. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–15, 2021. doi: 10.1145/3411764.3445518.

[20] Puntitra Sawadpong, Edward B. Allen, and Byron J. Williams. Exception handling defects: An empirical study. In *2012 IEEE 14th International Symposium on High-Assurance Systems Engineering*, pages 90–97, 2012. doi: 10.1109/HASE.2012.24.

[21] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html.

[22] Jairo Souza, Tales Alves, Robson Oliveira, Leopoldo Teixeira, and Baldoino Fonseca. Exception Miner: Multi-language static analysis tool to identify exception handling anti-patterns. In *Proceedings of the XXXVIII Brazilian Symposium on Software Engineering*, pages 741–747, 2024. doi: 10.5753/SBES.2024.3573. URL https://sol.sbc.org.br/index.php/sbes/article/view/30420.

[23] Jairo Souza, Tales Alves, Rodrigo Lima, Ericlecio Araújo, Leopoldo Teixeira, and Baldoino Fonseca. An empirical evaluation of using large language models to recommend exception handling tasks. In *Proceedings of the XL Brazilian Symposium on Software Engineering*, 2026. URL https://leopoldomt.github.io/assets/pdf/2026-sbes-jairo.pdf.

[24] Jairo Souza, Eric Coelho, João Correia, Rodrigo Lima, Leopoldo Teixeira, and Baldoino Fonseca. Exception handling bugs in Python: An empirical study of root causes, fix patterns, and anti-patterns. *Information and Software Technology*, 199:108264, 2026. doi: 10.1016/j.infsof.2026.108264. URL https://leopoldomt.github.io/assets/pdf/2026-ist.pdf.

[25] Rahul Yedida, Hong Jin Kang, Huy Tu, Xueqi Yang, David Lo, and Tim Menzies. How to find actionable static analysis warnings: A case study with FindBugs. *IEEE Transactions on Software Engineering*, 49: 2856–2872, 2023. doi: 10.1109/TSE.2023.3234206.

A Reproduction

There are two reproduction paths. They use different fixed versions and must not be mixed: the historical archive predates the current audit scripts, and the current scripts are not present at the historical commit.

A.1 Historical numerical archive (commit `3b4c567`)

Commit `3b4c567` (8 September 2026) contains the datasets, exports and scripts behind the reported counts, including the corrected Pylint invocation. It does *not* contain `tools/reproduce_paper.py`, `tools/reproduce_warning_coverage.py`, `tools/semgrep_diagnostic.py`, `paper/reference-audit.json` or `paper/annotation-corrections.json`. Several historical scripts write into tracked files under `bench/` and `paper/`; run them only in a disposable worktree, never in a working copy you want to keep:

```
git clone https://github.com/feiiiiii5/failroute && cd failroute
git worktree add ../failroute-3b4c567 3b4c567      # disposable tree
cd ../failroute-3b4c567
python -m venv .venv && . .venv/bin/activate
pip install '.[test,paper]'
python tools/fetch_corpus.py        # verifies 8 archives; rewrites the
                                    # lock's timestamp in this tree only
PYTHONPATH=src python -m pytest     # 221 tests; as root 220 pass, 1 skips
PYTHONPATH=src python tools/rescan_corpus.py --out /tmp/h-rescan --with-context
    # 476 findings (the detector at this commit is 0.9.1)
python tools/coverage_union.py --findings-dir bench/rescan-f2 \
    --out /tmp/h-union-621.json --emit-per-finding /tmp/h-621.jsonl
    # union 255/621; compare /tmp/h-621.jsonl with the committed export
python tools/coverage_union.py --findings-dir bench/rescan-v2 \
    --out /tmp/h-union-476.json --emit-per-finding /tmp/h-476.jsonl
    # union 252/476
cd - && git worktree remove --force ../failroute-3b4c567
```

Fetch the corpus into the worktree rather than symlinking it from another checkout: through a symlink, Pylint's parallel run stalled for over 20 minutes on one package that otherwise takes seconds. On 4 November 2026 this path gave 255/621 and 252/476, with the per-finding files and the rescan byte-identical to the committed exports. At that commit, `tools/rq5_refactor_delta.py` has no output option and writes `bench/rq5-refactor-delta.json`; inside the disposable tree that overwrite is harmless and can be compared with `git diff`.

A.2 Current audit

The current audit tooling was introduced in commit `b88cbad` (3 November 2026), extended in commit `3e19575` (whole-row comparison, a content check of the frozen artefacts and the separation of historical claims) and completed in commit `8040aeb` (4 November 2026), which makes the figure check compare drawn geometry as well as text; later commits do not change the audit tools. It adds a guard (`tools/_frozen_guard.py`) that makes the analysis scripts refuse to overwrite a git-tracked output unless `--allow-overwrite-frozen` is given; a fetch that reproduces the lock leaves the lock file untouched. One command runs every executable check into a new directory and compares each output with the frozen file:

```
git checkout 8040aeb
python -m venv .venv && . .venv/bin/activate
pip install '.[test,paper,paper-semgrep]'
python tools/fetch_corpus.py                    # lock unchanged if it matches
python -m pytest                       # 244 tests; as root 243 pass, 1 skips
python tools/reproduce_paper.py --out-dir /tmp/failroute-audit --with-semgrep
```

The steps are: corpus verification; the v0.8.0 scan from a disposable worktree at tag `v0.8.0`; the current scan; four-tool coverage on both frames; the frame delta recomputed from the fresh rows; the E722/BLE001

counts; the 68-probe matrix with parity check; the mapping-test replay; and, with `--with-semgrep`, the Semgrep diagnostic. Scan and coverage outputs are compared as whole JSON rows and checked for byte identity; the delta, E722 and probe outputs are compared as JSON with only timestamps and the interpreter path ignored. Every step except the last must report PASS; the Semgrep step reports PARTIAL while Semgrep records an analysis error. On 4 November 2026, on Linux x86_64 with Python 3.13, all eight non-Semgrep steps passed, with both scans and both coverage files byte-identical to the frozen exports, and the Semgrep step reported PARTIAL with the values in Section 4.2. The control corpus is checked separately with `PYTHONPATH=src python tools/control_rescan.py --check`, which passed for all eight packages. A separate check (`tools/check_frozen_truth.py --corpus`) verifies the protected inputs by SHA-256 and rebuilds `intervals.json` and the three figures. A rebuilt figure must carry the same text and the same drawn geometry (vector marks, marker positions and decoded image cells, relative to the axes area) within stated tolerances, so a moved point or cell is detected even when every label is unchanged. The committed figures were written by matplotlib 3.11.0 (Figures 1–2) and 3.11.1 (Figure 3) and are rebuilt with 3.11.1; the residual differences are at most 0.04% of the axes width or height for vector marks and 0.66% of image samples, and are printed by every run.

The repository's claim checks (`tools/verify_claims.py`) separate the checks the current version must pass from historical ones. Eight historical claims cannot be re-run from public artefacts: six need commits `6eb7973` or `50275e8`, which were lost when the history of one development batch was rewritten before publication, and two need an unreleased companion repository. They are reported as `HIST`, never as passing, with their expectations unchanged. For five of them, the same expectation is re-derived on the public commits that play the same roles (`06eba40`, `82734d1`), which shows that public code reproduces the recorded results but not that the trees are identical. Two further slow claims fail as expected: they demand an error-free five-tool Semgrep run, which the strict harness refuses (Section 4.2).

A.3 Execution notes

The historical corpus scans were run on macOS arm64; the CI matrix (Ubuntu, macOS and Windows; Python 3.9–3.13) runs the test suite, not the corpus scans. The re-execution of 3–4 November 2026 on Linux x86_64 reproduced the historical outputs exactly. The v0.8.0 scanner exits with status 1 when it reports findings even if the scan succeeded; the 0.9.x scanner separates severity and error exit statuses. Three pipeline scripts are deliberately not part of either path because they overwrite hand-curated artefacts: `draw_sample.py`, `merged_pr_recall.py` and `compare_linters_corpus.py`; the old `run_semgrep_baseline.py` reads the v0.7.0 frame and now refuses to run without the overwrite flag.

B Detector details

What counts as recording a failure. A handler is treated as signalling its failure if it re-raises; calls `exit` or `_exit`; calls `warnings.warn` or `warnings.warn_explicit`; or calls a logger-like name. The test is not uniform. A catch-all handler (bare `except:` or `except Exception:`) must call a method named `error`, `warning`, `warn`, `exception`, `critical` or `fatal`; a typed handler qualifies on *any* method of a logger-like object, `debug` and `info` included. A bare `print()` never qualifies. The asymmetry is deliberate (a typed clause documents an anticipated failure) but does not establish that a debug message reaches the caller. Only `silent-fallback` and `implicit-fallback` consult the test. "Logger-like" is a base-name list (`logger`, `logging`, `log`, `_logger`, `_log`, `sentry_sdk`) plus a suffix heuristic. Handler bodies are traversed through control flow but not into nested function, lambda or class bodies.

What counts as a fallback value. A fixed constant set (`None`, `False`, `0`, `0.0`, `""`, `[]`, `{}`, `()`), with `True`, `1` and `1.0` marked ambiguous, plus namespace-qualified sentinels such as `Status.UNKNOWN` and configured values. Bare names are never matched.

Exemptions. Handlers catching only `KeyboardInterrupt`, `SystemExit`, `GeneratorExit`, `StopIteration`, `StopAsyncIteration` or `CancelledError` are exempt (a tuple only if every member is); so are handlers containing `# failroute: ignore` or `# pragma: no cover`, and rules disabled in configuration. The exemption can hide a genuine mistake.

v0.9.1 refinement. Three heuristics replace the plain constant test: (1) *declared value range*: if the enclosing function's declared return type admits the substituted value, the finding is downgraded (return channel only, explicit annotations only); (2) *exception–predicate isomorphism*: whether the handler's work is structurally a `-> bool` predicate the codebase declares; (3) *escape channel*: whether the value is returned or assigned to object state. On the 476-finding frame the isomorphism layer classified no finding as isomorphic (254 non-isomorphic, 206 unknown, 16 without a handler), so it contributed nothing to the frame change. Severities: 208 HIGH, 126 MEDIUM, 129 INFO, 13 LOW. Each finding identifier is derived from the handler alone, so the two findings at a multi-target site share an identifier.

Tests and fixtures. v0.8.0 has 158 tests and a unit-fixture set (v6) of 36 positive and 32 negative cases; v0.9.1 has 221 tests and fixture set v7 with 34 positive and 34 negative cases (five narrow-type `contextlib.suppress` cases became negative and three other cases changed in the opposite direction). The current revision has 244 tests. Fixture agreement shows that the detector implements its specification, not that its findings are defects.

C Measurement corrections and revision history

This appendix records corrections made during the study so that earlier figures remain traceable.

Detector: implicit-fallback control flow (v0.7.0). All four false-positive labels in the 80-item sample were `implicit-fallback` findings with one root cause: `_has_top_level_terminal()` inspected only direct statements, so a `return` or `raise` nested in an exhaustive `if/else` was invisible. A recursive analysis covering `if/else`, `with`, `try/except/finally` and `match`, with six regression tests derived from the four shapes, reduced the rule's findings from 10 to 0 and the frame from 659 to 649. The v0.8.0 release removed 28 further findings through four audited fixes (tuple-typed handlers, `warnings.warn` as a signal, `StopAsyncIteration` exemption, optional-dependency probes): 649 → 621. No DEFECT-labelled site was removed.

Harness: Pylint read set. Described in Section 4.2. Under RQ1's narrow selections the 152 affected files yield 5, 0 and 0 messages for Ruff, Bandit and flake8 under either invocation; under full selections they yield 6,879, 104 and 2,375 messages, identical both ways. The control corpus has no `__init__`-less directory. Without `click` importable, Pylint reports 564 rather than 563 messages and the directory invocation hides 35 messages across 21 files rather than 34 across 20; co-location is unaffected.

Harness: Pylint message identifiers. Pylint accepts the retired `W0703` and reports its messages as `W0718`; over the corpus the selection emits 423 `W0718`, 134 `W0702` and 6 `W0706`. A reproducer counting `W0703` would find zero.

Baseline selection: BLE001. With `BLE001` added, Ruff reports 410 findings (76 without) and co-locates with 127 (72 without); the union stays at 255/621. The Pylint-to-Ruff co-location ratio is therefore 3.54 without `BLE001` and 2.01 with it. Ruff 0.16.0, 0.16.4 and 0.16.5 give set-identical hits for both selections; the committed artefact was produced with 0.16.4 and the paper extra pins 0.16.5.

Mapping: details of the correction. The 146 findings that lost credit were all `no-action` (93 INFO, 48 MEDIUM, 5 HIGH); 137 became predicted-unmatched and 9 kept a rule that genuinely fires. HIGH findings with no predicted coverage rose from 20 to 25. `pydantic_ai/_agent_graph.py:205`, an `except BaseException: pass`, illustrates why Ruff and Bandit need separate caught-type conditions. All 146 have a `pass` body, so the body-form condition changes no number in this corpus. The pre-correction export is preserved as `bench/corpus-coverage-per-finding-v2-pre-coverby-fix.jsonl`. The five findings that disagreed after the correction were `except Exception:` handlers in files under directories without `__init__.py`; parse failures, inline pragmas and the `as` binding were tested and ruled out before the read-set cause was found.

Supplementary baseline: Semgrep audit. The original paper extra did not install Semgrep; a pinned `paper-semgrep` extra was added in the 2026-11-03 audit. The strict harness stops on the parse error described in Section 4.2; `tools/semgrep_diagnostic.py` and `bench/semgrep-diagnostic-2026-11-03.json` record the bound. The older `tools/run_semgrep_baseline.py` read the v0.7.0 frame and overwrote frozen files.

Reference set and labels. The 2026-11-03 audit withdrew the pooled reference-set ratios (Section 5.2) and corrected one supplementary rationale: returning `None` from `JSONEncoder.default` produces JSON `null`, not a not-serialisable signal. The original label is preserved but not reaffirmed. Earlier versions of this manuscript reported a weighted projection of 130 LLM DEFECT labels (20.0%) in the 649-candidate frame; it is a projection of labels, not of defects, and is not used in the argument.

D Supplementary label analyses

All tables in this appendix describe LLM labels (Section 5.1). They are not defect-rate estimates.

Table 10: LLM labels in the original 80-finding sample, by rule.

Rule	n	DEFECT	CONTRACT	FALSE POS.
`silent-fallback`	49	12	37	0
`no-action`	23	0	23	0
`implicit-fallback`	4	0	0	4
`silent-suppress`	3	0	3	0
`masked-exception`	1	0	1	0
Total	80	12	64	4

Table 11: LLM labels in the original 80-finding sample, by package.

Package	n	DEFECT	CONTRACT	FALSE POS.
deepteam	14	12	1	1
inspect_ai	41	0	40	1
pydantic_ai_slim	11	0	10	1
trl	4	0	4	0
garak	3	0	3	0
fickling	3	0	2	1
smolagents	2	0	2	0
uqlm	2	0	2	0

Copied families. The 12 DEFECT labels fall into three families: (1) four handlers that catch an error from `model.get_system_prompt()` and assign an empty string, so evaluations may run without intended criteria (the author's issue on this pattern predates the labelling and is not independent validation); (2) five handlers that evaluate `self.score == 1`, discard the result and assign `self.success = False`, where the conversion mechanism is unestablished: observed values did not raise, the pinned function has no caller, and its state depends on an external base class; (3) three handlers that return `vulnerable = False` after an error while iterating results, a potentially fail-open security verdict.

Grouping sensitivity. Grouping copies by package and enclosing function gives 3 DEFECT groups of 68 (4.4%); by exact signature, 5 of 71 (7.0%); by package, file and function, 12 of 78 (15.4%). No grouping is established as the correct independent unit.

Handler forms. The 80 findings include 13 bare handlers (12 DEFECT), 18 `except Exception:` handlers, 46 narrowly typed handlers and 3 `contextlib.suppress` sites; none outside the bare group carries an original DEFECT label.

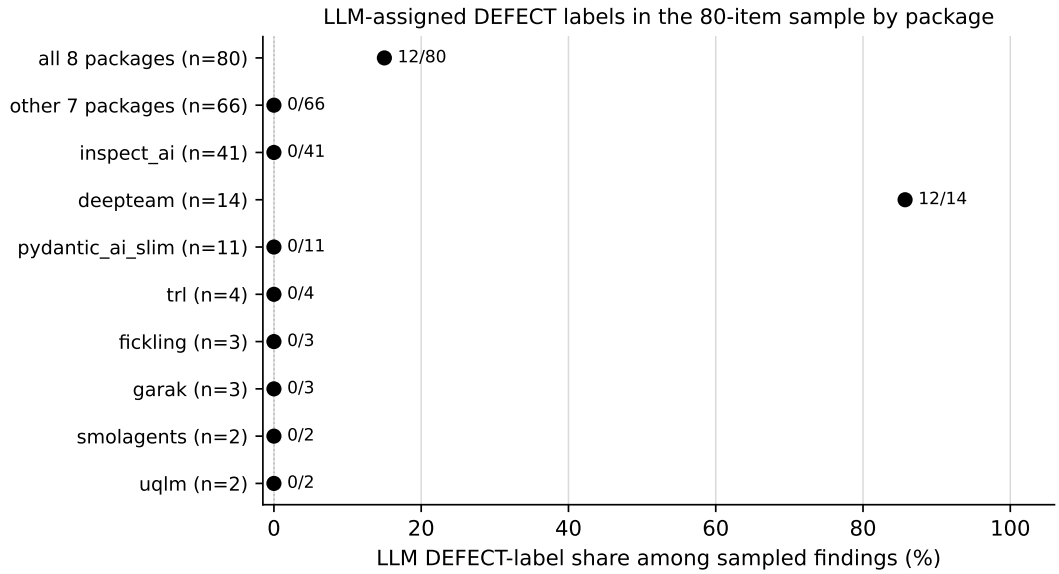


Figure 3: Share of sampled records assigned the LLM DEFECT label, by package, in the 80-item sample. Not a package-level defect rate.

Disposition under the v0.9.1 refinement. Joining all 140 labels to the refined frame on (package, file, line), deliberately excluding the rule name because one handler was reclassified between rules, 104 join and 36 do not; the 36 are 33 CONTRACT and 3 FALSE POSITIVE, all from the original 80, and all are deletions rather than line drift. All 13 DEFECT-labelled observations survive at HIGH. Of the 122 CONTRACT labels, 58 (47.5%) were dropped or reduced to INFO or LOW; among those in functions declaring a `None`-admitting return type, 35 of 38 (92.1%); for a literal `-> None`, 22 of 23. Fifty-five CONTRACT labels sit in functions without a declared return type, where the declared-range heuristic cannot apply. Two FALSE POSITIVE labels survive at HIGH (`pydantic_ai/ui/_event_stream.py:293`, reported twice, and `fickling/polyglot.py:150`, reclassified from `implicit-fallback` to `silent-fallback`). The predicted mapping and executed co-location agree on all 104 joined observations.

Supplementary 60. Written to `paper/annotations-v2-additions.csv`; the frozen `annotations.csv` and `annotations-second-pass.csv` are unchanged and hash-locked. Each row records its rationale, the enclosing signature and the detector's own severity and mapping, recorded after the rationale was written. One row is marked as the weakest of the 60. The single DEFECT label among unmatched observations (`garak/probes/dan.py:134`) is one LLM label from a purposive sample and shows only that the unmatched cell was not empty of DEFECT labels.

E Maintainer records

Table 12: Three evidence sources and the finite records each describes. They have different units and selection processes and are not combined.

Source	Question	Observed record
LLM labels	What did the LLM rounds label in the stratified sample?	12/80 DEFECT labels, all in one package
Tracked sites	What did maintainers change during the tracked window?	13/478 tracked sites classified FIXED
Submitted PRs	What happened to the project-selected PRs?	16 of 43 merged

Tracked sites. `paper/arm2-tracked-sites.csv` tracks 2,128 `try` sites across ten releases of each main-corpus package and labels 478 FIXED or SURVIVED, discarding 924 too young, 681 vanished and 45 mutated. Its criterion (no raise, log or warn) is weaker than the detector's: only 185 of the 478 carry a detector rule, and 59.4% of rows are in `deepteam`. None of the 13 FIXED sites traces to a PR from the project account. A SURVIVED site records inaction, not intent.

Submitted PRs. Of 43 project-selected, family-related PRs, 16 merged (`paper/merged-pr-recall.csv`; 8 outside the main corpus) and 27 closed unmerged (`paper/h1_arm3_classification.csv`): 19 self-closed, 1 bot-closed, 3 closed without reaching the merits, 2 accepted in substance elsewhere and 2 adverse on the merits. Duplicates mean the 43 are not 43 distinct candidate defects. None of the 16 merged changes maps to a tracked site. These are descriptive outcomes of a self-selected set, not an acceptance rate. The historical titles for `agent-sweep`#197 and `inspect_ai`#4908 do not match their final GitHub titles; repository and PR number identify each change.

F Artefact inventory

- `paper/corpus-lock.json`, `paper/corpus-lock-control.json` — pinned inputs with archive and tree hashes.
- `bench/rescan-f2/`, `bench/rescan-v2/` — the 621- and 476-finding frames; `bench/corpus-coverage-per-finding-{621,v2}.jsonl` — one row per finding with co-located tools and predicted coverage; `bench/corpus-coverage-union-621frame.json`, `bench/corpus-coverage-union-v2.json` — aggregates.
- `bench/corpus-coverage-union-ruff-ble001.json` — the symmetric `BLE001` variant; `bench/corpus-coverage-union-5tool.json` — historical five-tool export; `bench/semgrep-diagnostic-2026-11-03.json` — the Semgrep error and bound.
- `bench/lint-rule-mapping.json` — the 68-probe matrix; `bench/corpus-coverage-per-finding-v2-pre-coverby-fix.jsonl` — the pre-correction export; `bench/rq5-refactor-delta.json` — frame delta and label disposition; `bench/e722-baseline.json` — E722/BLE001 counts.
- `bench/control-rescan/` — control-corpus scans.
- `paper/sample.jsonl`, `paper/annotations.csv`, `paper/annotations-second-pass.csv`, `paper/annotations-v2-additions.csv` — draws and LLM labels; `paper/annotation-corrections.json` — rationale correction and hashes of the unchanged label files; `paper/intervals.json` — sample counts for Figure 3.
- `paper/merged-pr-recall.csv`, `paper/reference-audit.json`, `paper/h1_arm3_classification.csv`, `paper/arm2-tracked-sites.csv`, `paper/j1-trulens2653-prefix.py` — maintainer records and the current audit; historical classifications are superseded where the audit qualifies them.
- `paper/audit-rerun-2026-11-03.md` — the commands and outcomes of the re-execution of 3–4 November 2026.

Source audit of sixteen historically selected changes

Repository	PR	Audit status	Reason (abridged)
uqlm	#462	supported	Retry NaNs reach aggregates; fix warns and uses NaN-aware reductions.
inspect_ai	#5068	supported	Warning already exists; failure still becomes exists=True and continuation.
uqlm	#459	supported	Failed-judge NaNs poison aggregates; fix warns and excludes them.
PyRIT	#2466	supported	Missing factories silently filter selected techniques; fix raises.
PyRIT	#2460	excluded	Error becomes auth-disabled configuration; TypeScript, outside language scope.
uqlm	#450	supported	Discarded retry errors lead to empty output; local warning is relevant.
inspect_ai	#4906	excluded	RetryError becomes the underlying exception; failure already propagates.
inspect_evals	#2167	supported	Missing issue_id becomes score zero, with an answer-field diagnostic.
unstructured	#4438	supported	errors=ignore drops undecodable bytes; fix adds strict decoding and fallback.
PyRIT	#2309	uncertain	Unknown characters are dropped; operational-failure membership is uncertain.
trulens	#2653	location-only	Local pass-to-continue match does not identify the repaired range validation.
uqlm	#425	excluded	Division originally crashes; the patch introduces a uniform fallback.
uqlm	#418	uncertain	Implicit None later raises TypeError; success laundering is unestablished.
agent-sweep	#195	supported	Error is printed, then callers claim success; fix guards those claims.
agent-sweep	#197	supported	Unreadable input is silently skipped; local OSError/return warning is relevant.
inspect_ai	#4908	supported	Malformed grades become plausible first-letter grades; fix validates input.

Supported = broader example, not a confirmed uniform defect population.
Pooled detection and non-handler ratios withdrawn; see reference-audit.json.

Figure 4: Source-review dispositions of the sixteen historically selected merged changes (audit of 2026-11-03). These are audit statuses, not independent defect judgements or detection outcomes; pooled ratios are withdrawn.